

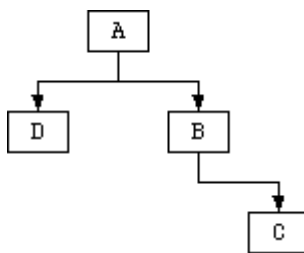
Hur används typkonvertering

Funktionen som används för konverteringen heter `dynamic_cast` och tar som parameter en datatyp till vilken man skall försöka konvertera och en pekare till ett objekt. Allmänt används `dynamic_cast` på följande sätt:

```
datatyp * variabel = dynamic_cast<datatyp *>(objekt);
```

Man placerar den datatyp till vilken man vill försöka konvertera ett objekt innanför `<>`, precis som då man använder templates (se [Kapitel 23](#)). En pekare till det objekt som man vill konvertera ges som parameter. `dynamic_cast` returnerar en pekare till konverterat objekt ifall konverteringen var möjlig och 0 ifall det inte var möjligt. Notera att pekaren ännu pekar på samma objekt, men det bara "tolkas" som ett objekt av en annan datatyp. Innehållet är dock detsamma. Vad kan man då konvertera? Man kan konvertera en pekare till en basklass till en pekare till en subclass, förutsatt att objektet från början skapats som ett objekt av subclassen. Man kan dock inte konvertera ett objekt till ett annat om det inte är en super- eller subclass. Man kan inte heller konvertera ett objekt som är skapat som ett objekt av en basklasstyp till en subklasstyp. Vi ser på den arvshierarki vi hade ovan och lägger till en klass `D` som ärver `A`:

Figur 26-1. Arvshierarki



Anta att vi har ett objekt som är skapat med följande anrop:

```
A * BasKlass = new A;
```

Detta objekt kan inte konverteras till någon annan klass. Om vi istället har följande objekt:

```
C * SubSub = new C;
```

och vi refererar till det från en metod som tagit en pekare till ett objekt av typen `A` som argument, så kan vi konvertera det till både `B` och `C`. Detta går eftersom ett objekt av typen `C` är (*is-a*) även ett objekt av typen `B` och ett av typen `A` (se [Kapitel 15](#)).

Om vi däremot har ett objekt av typen `D` som refereras via en pekare av typen `A` kan det *inte* konverteras till varken `B` eller `C`, eftersom ingendera klassen är relaterad till `D` på något annat sätt än att de har samma basklass `A`.

Praktiskt exempel

Vi kan definiera en metod som tar objekt av typen `A` på följande sätt:

```
void doIt (A * Data) {
    C * ObjC;
    B * ObjB;

    // försök konvertera till ett 'C'
    if ( ( ObjC = dynamic_cast<C *>(Data) ) != 0 ) {
        cout << "Objektet av typen C" << endl;
    }

    // ett 'B'?
    else if ( ( ObjB = dynamic_cast<B *>(Data) ) != 0 ) {
        cout << "Objektet av typen B" << endl;
    }

    // måste vara ett 'A'
    else {
        cout << "Objektet av typen A" << endl;
    }
}
```

Funktionen använder sig av `dynamic_cast` för att försöka konvertera till de olika subklasserna. Man måste börja med den mest ärvda klassen, annars får man kanske inte det man vill ha. Om vi t.ex. skickar ett objekt av typen `C` vill vi att det skall känas igen som ett `C` och ingenting annat. Men om vi testat om objektet är av typen `B` *före* vi testat om det är av typen `C` så kommer det att kunna konverteras till ett `B` (det *är* ju trots allt en subklass av `B`!), och således testas inte alls om det är av typen `C`. Vi kunde ju förstås lämna bort `else`-delarna ovan så skulle det problemet vara löst.

[Företgående](#)
Typinformation

[Hem](#)
[Upp](#)

[Nästa](#)
Typinformation